

Week 08: Classification and Evaluation: Defects, Failures and Grades

Artificial Intelligence Applications in Engineering (MUH-920 Mühendislikte Yapay Zeka Uygulamaları). Lecture notes

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 | utkukose.com | github.com/utkukose

utkukose@sdu.edu.tr | utku.kose@und.edu | ukose@up.edu.mx | utkukose@gmail.com

Week overview, study path and tasks: <https://github.com/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-08/README.md>

Lecture page with animations and knowledge checks: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-08/lecture.html>

Interactive lab with self-assessment and reflection: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-08/lab.html>

Colab notebook with Python step 8 and the hands-on work:

https://colab.research.google.com/github/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-08/NB08_classification_evaluation.ipynb

Content licensed under CC BY 4.0. This course is updated in line with current developments in the field.

Last update: October 2026.

Overview

Many engineering decisions are categorical: accept or reject a part, which fault mode is developing, which variety a grain belongs to, whether a rock burst is likely. Classification learns such decisions from labelled examples. This week compares logistic regression, nearest neighbours, support vector machines and tree ensembles [1, 2], and spends as much time on evaluation as on models: confusion matrices, precision and recall, ROC and precision-recall curves, thresholds chosen by costs, and the treatment of rare failures [3, 4, 5]. The core cases are the dry bean dataset, produced with computer vision in Türkiye [6, 7], and the AI4I predictive maintenance dataset [8, 9].

Estimated study time: 9 to 11 hours.

Learning outcomes

By the end of the week, students are expected to train and compare multiclass classifiers with scikit-learn pipelines and stratified splits, to compute and interpret a confusion matrix, precision, recall, F1 and their macro averages, to draw and read ROC and precision-recall curves, to choose a decision threshold from engineering costs, to handle class imbalance with class weights or resampling, and to explain a model's behaviour with permutation importance.

Python in this week

These notes close with Python step 8: Grouping, counting and evaluation functions. It covers counting and grouping with pandas, boolean logic on columns, functions that return several values, and results collected in tables, applied to a machine failure log [10]. In the course materials, the step is taught at the

start of the Colab notebook, where each snippet runs in a cell next to its explanation and short exercises give immediate feedback, and later sections of the notebook mark further constructs as Python moves.

Classification problems in engineering

A classifier assigns an input to one of several classes. Quality inspection separates good from defective parts, condition monitoring distinguishes fault modes, food engineering grades products, and geology assigns facies to depth intervals of a well. A classifier divides the input space into regions separated by decision boundaries. Most modern classifiers do more than name a class: They estimate a probability or a score for each class, and a decision rule, usually a threshold, turns the score into an action. Separating the model from the decision is essential in engineering, because the costs of different mistakes are rarely equal.

Formulas: Scores, probabilities and decisions

A probabilistic classifier estimates for an input x the probability $\hat{p}(x) = \hat{P}(y=1|x)$ of the positive class. A threshold t turns it into a decision:

$$\hat{y} = 1 \text{ if } \hat{p}(x) \geq t, \hat{y} = 0 \text{ otherwise}$$

The decision boundary is the set of inputs at which the decision changes, $\{x : \hat{p}(x) = t\}$. With K classes, the classifier estimates $\hat{p}_k(x)$ for every class k , and the usual rule picks the class with the largest probability, $\hat{y} = \arg \max_k \hat{p}_k(x)$. The model supplies $\hat{p}$; the threshold belongs to the decision and is set from the costs of the errors, as the section on curves and costs shows.

Models for classification

Logistic regression models the log-odds of a class as a linear function of the inputs, so its decision boundary is a straight line or a plane, and its coefficients show the direction in which each input pushes the decision [11]. The k-nearest-neighbour classifier takes a vote among the most similar training examples [12]. Support vector machines find the boundary with the largest margin between classes and use kernels to draw curved boundaries in the original space [1]. Decision trees ask a sequence of threshold questions and are easy to read [13, 14]; random forests and gradient boosting combine many trees and are among the strongest methods for tabular engineering data [2, 15].

Animation: Decision boundaries of three classifiers. The same two-class data are separated by logistic regression, k nearest neighbours and a decision tree. Change k and the depth and watch the boundary move between too rigid and too wiggly. Run it on the [lecture page](#).

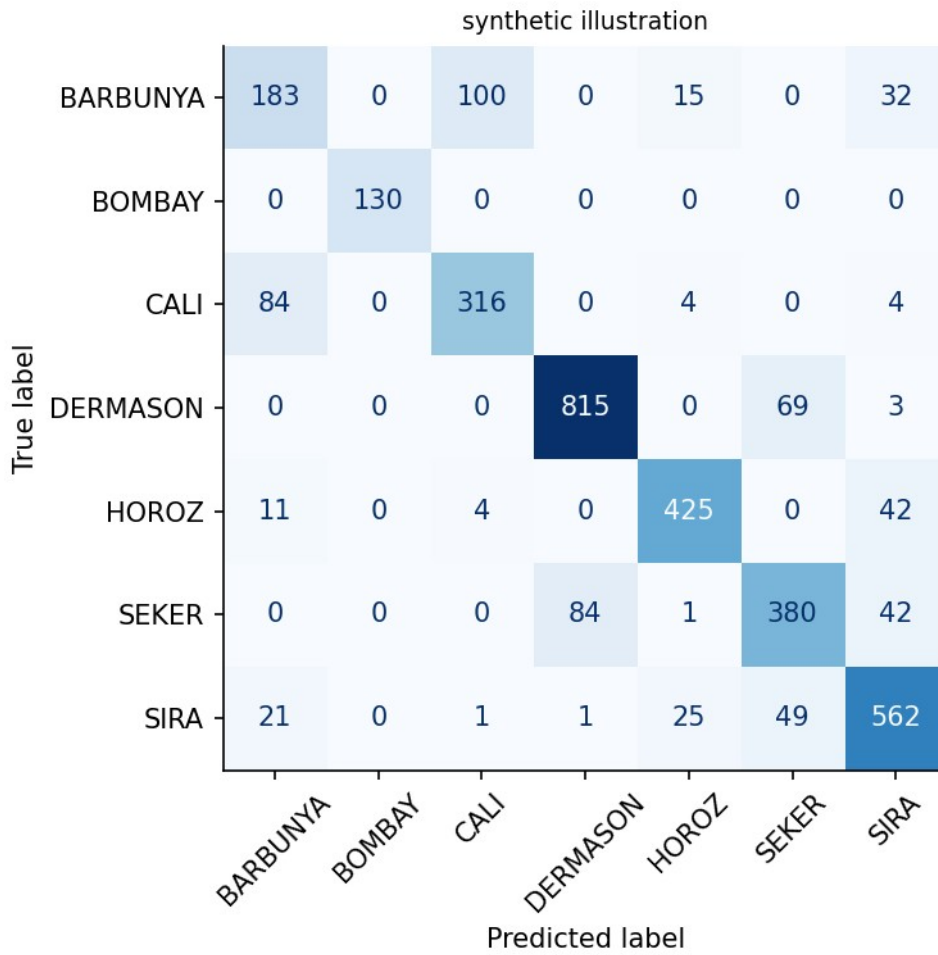


Figure 8.1. Confusion matrix of a random forest on the synthetic stand-in of the dry bean data. Most errors occur between classes with similar shapes.

Check your understanding. Which classifier draws a linear decision boundary in the original input space?

- A. Logistic regression without feature transformations
- B. k nearest neighbours with k = 1
- C. A random forest
- D. A support vector machine with a radial basis function kernel

Answer: A. Logistic regression models the log-odds linearly, so the boundary where the probability equals the threshold is a hyperplane.

Formulas: Logistic regression, impurity and the margin of a support vector machine

Logistic regression passes a linear score through the logistic function, which maps any real number to a probability [11]:

$$z = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p, \hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Solving for z shows that the score is the logarithm of the odds, $z = \ln(\hat{p}/(1 - \hat{p}))$, so a coefficient β_j multiplies the odds by e^{β_j} for every unit of x_j . The coefficients minimise the log-loss, also called the cross-entropy, of the training examples, whose gradient has a simple form:

$$L = -[y \ln \hat{p} + (1 - y) \ln (1 - \hat{p})], \frac{\partial L}{\partial \beta_j} = (\hat{p} - y) x_j$$

with $x_0 = 1$ for the intercept. Gradient descent repeats $\beta_j \leftarrow \beta_j - \eta \partial L / \partial \beta_j$ with a learning rate η .

Decision trees choose the split that makes the children purest. With the class shares p_k in a node, the Gini impurity and the entropy are [13, 14]

$$G = 1 - \sum_k p_k^2, H = - \sum_k p_k \log_2 p_k$$

and a split into children with n_L and n_R of the n examples is scored by the weighted impurity of the children; for the entropy, the decrease is the information gain:

$$IG = H_{\text{parent}} - \frac{n_L}{n} H_L - \frac{n_R}{n} H_R$$

Support vector machines separate two classes, coded $y_i = \pm 1$, by the hyperplane $w \cdot x + b = 0$ with the widest margin. Requiring $y_i(w \cdot x_i + b) \geq 1$ for every training example, the margin between the two classes is $2/\|w\|$, so the machine minimises $\|w\|^2/2$ under these constraints [1]. Kernels such as the radial basis function $K(x, x') = e^{-\gamma \|x - x'\|^2}$ replace dot products and give curved boundaries. The nearest-neighbour classifier of Week 7 takes a majority vote among the k closest training examples [12].

Worked example: A logistic model of machine failure

A failure model uses two standardised inputs, torque x_1 and tool wear x_2 , with $\beta_0 = -3$, $\beta_1 = 1.5$ and $\beta_2 = 1$. A machine has $x_1 = 1.2$ and $x_2 = 0.8$.

Step 1. Score and probability.

$$z = -3 + 1.5 \times 1.2 + 1 \times 0.8 = -3 + 1.8 + 0.8 = -0.4$$

$$\hat{p} = \frac{1}{1 + e^{0.4}} = \frac{1}{1 + 1.491825} = 0.401312$$

Step 2. Odds. One standard deviation more torque multiplies the odds of failure by $e^{1.5} = 4.4817$.

Step 3. Loss. If the machine failed ($y = 1$), $L = -\ln 0.401312 = 0.913016$; if it did not ($y = 0$), $L = -\ln(1 - 0.401312) = 0.513015$.

Step 4. One gradient step. The machine did fail. With $\hat{p} - y = 0.401312 - 1 = -0.598688$ the gradient is

$$-0.598688 \times (1, 1.2, 0.8) = (-0.598688, -0.718426, -0.47895)$$

and with $\eta = 0.5$:

$$\beta_0 = -3 - 0.5 \times (-0.598688) = -2.700656$$

$$\beta_1 = 1.5 - 0.5 \times (-0.718426) = 1.859213, \beta_2 = 1 - 0.5 \times (-0.47895) = 1.239475$$

Step 5. The effect. The new score is

$$-2.700656 + 1.859213 \times 1.2 + 1.239475 \times 0.8 = -2.700656 + 2.231056 + 0.99158 = 0.52198, \text{ so}$$

$\hat{p} = 1/(1 + e^{-0.52198}) = 0.62761$ and the loss falls from 0.913016 to $-\ln 0.62761 = 0.465836$. Part C of the [interactive lab](#) repeats such steps over a whole data set.

Worked example: Gini impurity and information gain of a split

A node holds 40 machines, 10 of which failed. A split at a torque of 50 Nm sends 22 machines with 2 failures to the left and 18 machines with 8 failures to the right.

Step 1. The parent. With $p = 10/40 = 0.25$:

$$G = 1 - (0.25^2 + 0.75^2) = 1 - (0.0625 + 0.5625) = 0.375$$

$$H = -(0.25 \log_2 0.25 + 0.75 \log_2 0.75) = -(0.25 \times (-2) + 0.75 \times (-0.415037)) = 0.811278$$

Step 2. The children. Left, with $p = 2/22 = 0.090909$ and $1 - p = 0.909091$; right, with $p = 8/18 = 0.444444$ and $1 - p = 0.555556$:

$$G_L = 1 - (0.090909^2 + 0.909091^2) = 0.165289, G_R = 1 - (0.444444^2 + 0.555556^2) = 0.493827$$

$$H_L = -(0.090909 \times (-3.459432) + 0.909091 \times (-0.137504)) = 0.439497$$

$$H_R = -(0.444444 \times (-1.169925) + 0.555556 \times (-0.847997)) = 0.991076$$

Step 3. Weighted impurity and gain.

$$G_{\text{split}} = \frac{22}{40} \times 0.165289 + \frac{18}{40} \times 0.493827 = 0.313131, \Delta G = 0.375 - 0.313131 = 0.061869$$

$$H_{\text{split}} = 0.55 \times 0.439497 + 0.45 \times 0.991076 = 0.687708, IG = 0.811278 - 0.687708 = 0.12357$$

The tree compares these numbers over all candidate thresholds and features and keeps the best split. Part D of the [interactive lab](#) runs this search.

Worked example: The margin of a separating line

Two positive examples, $(3, 3)$ and $(4, 2)$, and two negative ones, $(1, 1)$ and $(2, 0)$, are separated by $w = (0.5, 0.5)$ and $b = -2$.

Step 1. The constraints. For the positives and the negatives:

$$0.5 \times 3 + 0.5 \times 3 - 2 = 1, 0.5 \times 4 + 0.5 \times 2 - 2 = 1$$

$$0.5 \times 1 + 0.5 \times 1 - 2 = -1, 0.5 \times 2 + 0.5 \times 0 - 2 = -1$$

Every product $y_i(w \cdot x_i + b)$ equals 1, so all four points lie on the edges of the margin and are support vectors.

Step 2. The margin. $\|w\| = \sqrt{0.5^2 + 0.5^2} = \sqrt{0.5} = 0.707107$, so the margin is $2/0.707107 = 2.828427$, the distance between the lines $x_1 + x_2 = 6$ and $x_1 + x_2 = 2$.

Step 3. A kernel value. With $\gamma = 0.5$, the points $(3, 3)$ and $(1, 1)$ have $\|x - x'\|^2 = 2^2 + 2^2 = 8$ and $K = e^{-0.5 \times 8} = e^{-4} = 0.018316$: Distant points barely interact.

The confusion matrix and its metrics

For a binary problem with a positive class, such as “failure”, the confusion matrix counts true positives (TP), false positives (FP), false negatives (FN) and true negatives (TN). Accuracy, the share of correct decisions, is misleading when classes are imbalanced: If 3 percent of machines fail, a model that always predicts “no failure” is 97 percent accurate and useless. Precision, $TP / (TP + FP)$, is the share of alarms that are real. Recall, $TP / (TP + FN)$, is the share of real failures that are caught. The F1 score is their harmonic mean. For several classes, the metrics are computed per class and averaged, either equally for every class (macro average) or weighted by class size (weighted average); the macro average reveals poor performance on small classes.

Check your understanding. A model raises 50 alarms, of which 40 are real failures, and misses 10 failures. What are its precision and recall?

- A. Precision 0.8 and recall 0.8
- B. Precision 0.8 and recall 0.4
- C. Precision 0.4 and recall 0.8
- D. Precision 1.0 and recall 0.8

Answer: A. Precision is $40 / 50 = 0.8$ and recall is $40 / (40 + 10) = 0.8$.

Formulas: Metrics of the confusion matrix

With the counts TP, FP, FN and TN of a binary classifier [3]:

$$\text{accuracy} = \frac{TP + TN}{TP + FP + FN + TN}, \text{precision} = \frac{TP}{TP + FP}, \text{recall} = \frac{TP}{TP + FN}$$

$$\text{specificity} = \frac{TN}{TN + FP}, F_1 = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

Recall is also called the true positive rate and specificity the true negative rate. F_1 is the harmonic mean of precision and recall, which stays low unless both are high. With K classes, each class k is in turn treated as positive, and its scores are averaged either equally or weighted by the numbers n_k of examples of each class:

$$F_1^{\text{macro}} = \frac{1}{K} \sum_{k=1}^K F_{1,k}, F_1^{\text{weighted}} = \frac{1}{n} \sum_{k=1}^K n_k F_{1,k}$$

Worked example: Alarms for 1000 machines

A model raised 50 alarms for 1000 machines; 40 of them were real failures, and 10 failures were missed. So TP = 40, FP = 10, FN = 10 and TN = 1000 - 40 - 10 - 10 = 940.

Step 1. The metrics.

$$\text{accuracy} = \frac{40 + 940}{1000} = 0.98, \text{precision} = \frac{40}{40 + 10} = 0.8, \text{recall} = \frac{40}{40 + 10} = 0.8$$

$$\text{specificity} = \frac{940}{940 + 10} = 0.989474, F_1 = \frac{2 \times 0.8 \times 0.8}{0.8 + 0.8} = 0.8$$

Step 2. A useless model. A model that never raises an alarm has TP = 0, FN = 50, FP = 0 and TN = 950: Its accuracy is $950 / 1000 = 0.95$, while its recall is $0 / 50 = 0$. Accuracy hides the rare class.

Worked example: Three classes, macro and weighted averages

A classifier of 100 machine states gives the following confusion matrix, with the actual classes in the rows and the predicted classes in the columns:

Actual \ predicted	normal	wear	overheat	Total
normal	45	3	2	50
wear	5	22	3	30
overheat	2	3	15	20

The column totals are 52, 28 and 20.

Class normal. Precision $45 / 52 = 0.865385$, recall $45 / 50 = 0.9$,
 $F_1 = 2 \times 0.865385 \times 0.9 / (0.865385 + 0.9) = 0.882353$.

Class wear. Precision $22/28=0.785714$, recall $22/30=0.733333$,
 $F_1=2 \times 0.785714 \times 0.733333 / (0.785714 + 0.733333) = 0.75862$.

Class overheat. Precision $15/20=0.75$, recall $15/20=0.75$, $F_1=2 \times 0.75 \times 0.75 / (0.75 + 0.75) = 0.75$.

Averages. The accuracy is $(45+22+15)/100=0.82$. The macro average is
 $(0.882353+0.75862+0.75)/3=0.796991$, and the weighted average is
 $(50 \times 0.882353 + 30 \times 0.75862 + 20 \times 0.75)/100 = (44.11765 + 22.7586 + 15)/100 = 0.818762$. The
macro average is lower, because the two small classes are recognised worse than the large one. Part B of
the [interactive lab](#) computes these metrics for any matrix.

Curves, thresholds and costs

Varying the threshold traces curves. The ROC curve plots the true positive rate against the false positive rate, and the area under it (AUC) equals the probability that a random positive example receives a higher score than a random negative one [3]. When positives are rare, the precision-recall curve is more informative, because the false positive rate can look small while alarms are still mostly false [4].

The threshold should come from the application. If a missed bearing failure costs twenty times more than an unnecessary inspection, the expected cost for each threshold on validation data can be computed, and the threshold with the lowest cost chosen. The default of 0.5 is only correct when both errors cost the same and the probabilities are well calibrated.

Check your understanding. Positives are 1 percent of the data. Which curve is recommended to compare classifiers?

- A. The ROC curve only
- B. The precision-recall curve
- C. The training loss curve
- D. The learning rate curve

Answer: B. With rare positives, precision reveals how many alarms are false, which the false positive rate hides [4].

Formulas: ROC curve, AUC and the cost of a threshold

Lowering the threshold t from above the largest score to below the smallest moves a classifier through all its operating points. At each threshold, the true positive rate and the false positive rate are [3]

$$\text{TPR}(t) = \frac{\text{TP}(t)}{n_p}, \text{FPR}(t) = \frac{\text{FP}(t)}{n_N}$$

where n_p and n_N are the numbers of positive and negative examples. The ROC curve joins the points (FPR, TPR) , and the area under it follows from the trapezoid rule over consecutive points:

$$\text{AUC} = \sum_i (\text{FPR}_{i+1} - \text{FPR}_i) \frac{\text{TPR}_i + \text{TPR}_{i+1}}{2}$$

The same number is the share of positive and negative pairs in which the positive example has the higher score, with ties counted as one half:

$$\text{AUC} = \frac{1}{n_p n_N} \sum_i \sum_j ([s_i > s_j] + 1/2 [s_i = s_j])$$

The first sum runs over the positive examples i , the second over the negative examples j , and the bracket [...] is 1 when the condition holds and 0 otherwise. The precision-recall curve plots precision against recall instead, which shows the share of false alarms directly when positives are rare [4]. With costs c_{FN}

for a missed positive and c_{FP} for a false alarm, the cost of a threshold on validation data is $C(t) = c_{FN} FN(t) + c_{FP} FP(t)$. For a calibrated probability p , raising an alarm costs $(1 - p)c_{FP}$ on average and staying silent costs $p c_{FN}$, so an alarm pays off when

$$p \geq t^* = \frac{c_{FP}}{c_{FP} + c_{FN}}$$

Worked example: A ROC curve from ten scores

Ten validation cases have the scores and labels (P for failure, N for normal) in the table, sorted by score; $n_P = 4$ and $n_N = 6$. Each row lowers the threshold to the score of that case:

Threshold	Label	TP	FP	TPR	FPR	Precision
0.95	P	1	0	1/4 = 0.25	0/6 = 0	1/1 = 1
0.85	P	2	0	2/4 = 0.5	0/6 = 0	2/2 = 1
0.8	N	2	1	2/4 = 0.5	1/6 = 0.1667	2/3 = 0.6667
0.7	P	3	1	3/4 = 0.75	1/6 = 0.1667	3/4 = 0.75
0.6	N	3	2	3/4 = 0.75	2/6 = 0.3333	3/5 = 0.6
0.5	N	3	3	3/4 = 0.75	3/6 = 0.5	3/6 = 0.5
0.45	P	4	3	4/4 = 1	3/6 = 0.5	4/7 = 0.5714
0.3	N	4	4	4/4 = 1	4/6 = 0.6667	4/8 = 0.5
0.2	N	4	5	4/4 = 1	5/6 = 0.8333	4/9 = 0.4444
0.1	N	4	6	4/4 = 1	6/6 = 1	4/10 = 0.4

Step 1. Trapezoids. Only the steps in which FPR grows add area, and each of them widens FPR by 1/6. The first lies at TPR = 0.5, the next two at 0.75 and the last three at 1:

$$\frac{1}{6} \times \frac{0.5+0.5}{2} = \frac{1}{12}, \frac{1}{6} \times \frac{0.75+0.75}{2} = \frac{1}{8}, \frac{1}{6} \times \frac{1+1}{2} = \frac{1}{6}$$

$$AUC = \frac{1}{12} + 2 \times \frac{1}{8} + 3 \times \frac{1}{6} = \frac{2}{24} + \frac{6}{24} + \frac{12}{24} = \frac{20}{24} = 0.833333$$

Step 2. Pairs. There are $4 \times 6 = 24$ pairs. The failure with score 0.95 outranks all 6 normal cases, the one with 0.85 also 6, the one with 0.70 outranks 5 (all but 0.80), and the one with 0.45 outranks 3. Hence $AUC = (6+6+5+3)/24 = 20/24 = 0.833333$, the same value.

Step 3. Costs. A missed failure costs $c_{FN} = 20$ and a false alarm $c_{FP} = 1$. At $t = 0.5$ one failure is missed and three false alarms are raised: $C = 20 \times 1 + 1 \times 3 = 23$. At $t = 0.45$ no failure is missed and three false alarms remain: $C = 0 + 3 = 3$. At $t = 0.1$ every case is an alarm: $C = 0 + 6 = 6$. The cheapest threshold here is 0.45. For calibrated probabilities the formula gives $t^* = 1/(1+20) = 0.047619$, far below the default of 0.5. Part E of the [interactive lab](#) builds the table, the curve and the costs for any scores.

Imbalanced data

Rare events are the rule in engineering: failures, defects, accidents, rock bursts. Several remedies exist. Stratified splits keep the class proportions equal in training and test data. Class weights make errors on the rare class more expensive during training. Resampling duplicates rare examples or removes common ones, and SMOTE creates synthetic minority examples by interpolating between neighbouring minority samples [5]. None of these remedies creates information; the most effective step is often better features, more minority data or a problem definition that matches the decision.

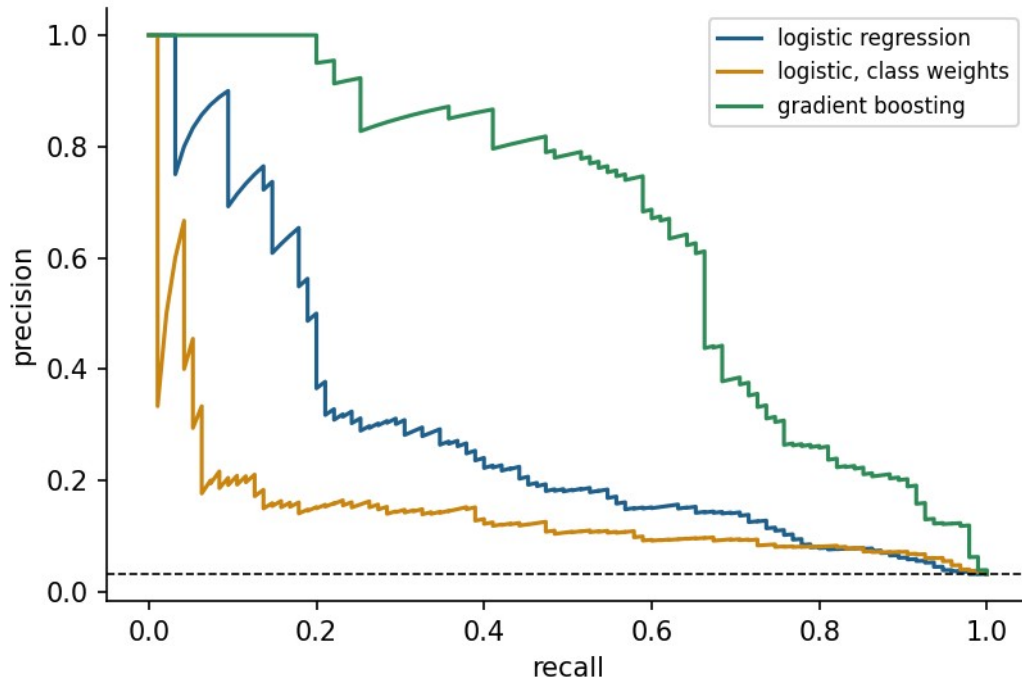


Figure 8.2. Precision-recall curves of three classifiers for rare machine failures on the synthetic stand-in of the AI4I data. The dashed line marks the share of failures, the precision of random guessing.

Formulas: Class weights, stratification and SMOTE

With n examples, K classes and n_k examples of class k , balanced class weights make every class contribute the same total weight to the loss:

$$w_k = \frac{n}{K n_k}$$

A stratified split keeps the class shares: A test share q receives about $q n_k$ examples of every class. SMOTE creates a synthetic minority example on the segment between a minority example x_i and one of its k nearest minority neighbours x_{nn} , with a random number λ between 0 and 1 [5]:

$$x_{\text{new}} = x_i + \lambda (x_{nn} - x_i)$$

Worked example: Rare failures in a milling data set

The AI4I 2020 data describe 10000 machining runs, of which 339 ended in a failure [9].

Step 1. Class weights. $w_{\text{failure}} = 10000 / (2 \times 339) = 10000 / 678 = 14.749263$ and $w_{\text{normal}} = 10000 / (2 \times 9661) = 10000 / 19322 = 0.517545$. Each failure counts about $14.749263 / 0.517545 = 28.5$ times as much as a normal run, and both classes reach the total weight $339 \times 14.749263 = 9661 \times 0.517545 = 5000$.

Step 2. Stratified split. A test share of 20 percent receives $0.2 \times 339 = 67.8$, that is about 68 failures, among 2000 runs; an unstratified random split could by chance give noticeably fewer.

Step 3. One synthetic failure. A failure with torque 62 Nm and tool wear 210 min has a neighbouring failure at 58 Nm and 190 min. With $\lambda = 0.4$:

$$x_{\text{new}} = (62, 210) + 0.4 \times (58 - 62, 190 - 210) = (62, 210) + (-1.6, -8) = (60.4, 202)$$

The new point lies between two real failures and adds no new information about where failures occur; it only changes how strongly the existing ones pull the model. Part F of the [interactive lab](#) generates such points.

Explaining classifiers

Engineers need to know why a model decides. Permutation importance measures how much a performance metric drops when the values of one feature are shuffled, which breaks its link to the target. SHAP values distribute a single prediction among the features according to a game-theoretic rule and are widely used for tree ensembles [16]. In predictive maintenance, explanations connect model output to physical mechanisms such as heat dissipation or tool wear [8]. Newer methods study the geometry of a model's decision function, for example GEMEX, which the instructor developed as an open Python package [17]. Explanations are models of models and can mislead, so they should be checked against domain knowledge and, where possible, against deliberately constructed test cases.

Check your understanding. Shuffling the values of “torque” in the test data lowers the F1 score of a failure classifier from 0.82 to 0.41. What does this indicate?

- A. Torque is unimportant for the model
- B. The model relies strongly on torque for its decisions
- C. Torque causes failures in every case
- D. The test data are leaked

Answer: B. A large drop after permutation shows that the model uses the feature; it does not prove a causal mechanism.

Formulas: Permutation importance and Shapley values

Permutation importance compares the score s of a model on intact data with the scores $s_{k,j}$ obtained after the values of feature j have been shuffled, in K repetitions:

$$I_j = s - \frac{1}{K} \sum_{k=1}^K s_{k,j}$$

Shapley values divide one prediction among the features. If $v(S)$ is the prediction when only the features in the set S are known, feature j receives the weighted average of its marginal contributions, where the sum runs over all sets S of the other features, including the empty set, among M features [16]:

$$\phi_j = \sum_S \frac{|S|!(M-|S|-1)!}{M!} (v(S \cup \{j\}) - v(S))$$

The exclamation mark denotes the factorial of Week 2. The values add up exactly to the difference between the prediction and the baseline, $\sum_j \phi_j = v(F) - v(\{\})$, which is why they are called additive explanations.

Worked example: Permutation importance of torque

A failure classifier reaches $F_1 = 0.82$. Shuffling torque three times gives 0.41, 0.45 and 0.39, with the mean $(0.41 + 0.45 + 0.39)/3 = 0.4167$, so the importance is $0.82 - 0.4167 = 0.4033$. Without the information in torque, the model loses about half of its F_1 , which agrees with the physics of tool failure under high load [8].

Worked example: Shapley values for three features

A model predicts the failure probability of one machine from torque (T), tool wear (W) and air temperature (A). The table gives the prediction $v(S)$ when only the features in S are known and the others are averaged out:

Known features S	$v(S)$
{ }	0.1
{ T }	0.4
{ W }	0.25
{ A }	0.12
{ T, W }	0.8
{ T, A }	0.45
{ W, A }	0.28
{ T, W, A }	0.85

With $M=3$, the weights are $\frac{0!2!}{3!}=\frac{1}{3}$ for the empty set, $\frac{1!1!}{3!}=\frac{1}{6}$ for sets of one feature and $\frac{2!0!}{3!}=\frac{1}{3}$ for sets of two features.

Step 1. Torque. The marginal contributions of T are $v(\{T\}) - v(\{\}) = 0.4 - 0.1 = 0.3$, $v(\{T, W\}) - v(\{W\}) = 0.8 - 0.25 = 0.55$, $v(\{T, A\}) - v(\{A\}) = 0.45 - 0.12 = 0.33$ and $v(\{T, W, A\}) - v(\{W, A\}) = 0.85 - 0.28 = 0.57$:

$$\phi_T = 1/3 \times 0.3 + 1/6 \times 0.55 + 1/6 \times 0.33 + 1/3 \times 0.57$$

$$\phi_T = \frac{2 \times 0.3 + 0.55 + 0.33 + 2 \times 0.57}{6} = \frac{2.62}{6} = 0.436667$$

Step 2. Tool wear. The contributions are $0.25 - 0.1 = 0.15$, $0.8 - 0.4 = 0.4$, $0.28 - 0.12 = 0.16$ and $0.85 - 0.45 = 0.4$:

$$\phi_W = 1/3 \times 0.15 + 1/6 \times 0.4 + 1/6 \times 0.16 + 1/3 \times 0.4$$

$$\phi_W = \frac{2 \times 0.15 + 0.4 + 0.16 + 2 \times 0.4}{6} = \frac{1.66}{6} = 0.276667$$

Step 3. Air temperature. The contributions are $0.12 - 0.1 = 0.02$, $0.45 - 0.4 = 0.05$, $0.28 - 0.25 = 0.03$ and $0.85 - 0.8 = 0.05$:

$$\phi_A = 1/3 \times 0.02 + 1/6 \times 0.05 + 1/6 \times 0.03 + 1/3 \times 0.05$$

$$\phi_A = \frac{2 \times 0.02 + 0.05 + 0.03 + 2 \times 0.05}{6} = \frac{0.22}{6} = 0.036667$$

Step 4. Check. $\frac{2.62 + 1.66 + 0.22}{6} = \frac{4.5}{6} = 0.75 = 0.85 - 0.1$, the difference between the prediction and the baseline. Of the raised risk, torque explains $0.436667/0.75 = 58$ percent, tool wear $0.276667/0.75 = 37$ percent and air temperature $0.036667/0.75 = 5$ percent. Part G of the [interactive lab](#) computes Shapley values for any table.

Python step 8: Grouping, counting and evaluation functions

Counting and grouping

Classification data raise two questions at once: How many examples does each class have, and how do the classes differ? `value_counts` counts the occurrences of each value in a column. `groupby` splits a table by the values of one column, applies a summary to each group and combines the results. The

practice log below lists invented machine records with the product type, the tool wear, the torque and whether a failure occurred.

```
import numpy as np
import pandas as pd

log = pd.DataFrame({
    "type": ["L", "M", "H", "L", "L", "M", "H", "L", "M", "L", "H",
    "M"],
    "tool_wear_min": [12, 180, 205, 221, 64, 230, 15, 199, 98, 240,
    150, 30],
    "torque_Nm": [38.2, 45.1, 61.0, 52.3, 40.0, 58.4, 33.5, 49.9,
    41.2, 63.8, 60.5, 36.1],
    "failure": [0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0],
})
print(log["type"].value_counts())
print(log.groupby("type")["failure"].mean().round(2))
print(log.groupby("failure")[["tool_wear_min",
"torque_Nm"]].mean().round(1))
```

Output

```
type
L    5
M    4
H    3
Name: count, dtype: int64
type
H    0.33
L    0.60
M    0.25
Name: failure, dtype: float64
      tool_wear_min  torque_Nm
failure
0                78.4       42.1
1               219.0       57.1
```

A rule as a classifier

A simple rule predicts a failure when the tool wear exceeds 200 minutes or the torque exceeds 60 N m. The combined condition produces a boolean Series, and `astype(int)` turns it into zeros and ones. `pd.crosstab` counts the combinations of actual and predicted classes, which is the confusion matrix of this week.

```
pred = ((log["tool_wear_min"] > 200) | (log["torque_Nm"] >
60)).astype(int)
actual = log["failure"]
print(pd.crosstab(actual, pred, rownames=["actual"],
colnames=["predicted"]))
```

Output

```
predicted  0  1
actual
0           6  1
1           1  4
```

Functions that return several values

A function can return several values separated by commas. Python packs them into a tuple, and the caller unpacks them into separate names. The function below counts true positives, false positives, false negatives and true negatives. A loop over several wear limits collects one dictionary per limit in a list, and `pd.DataFrame` turns the list into a table of results, a pattern that also serves to compare models, settings or datasets.

```
def confusion(y_true, y_pred):
    """Return the counts (tp, fp, fn, tn) for labels coded as 0 and 1."""
    y_true, y_pred = np.asarray(y_true), np.asarray(y_pred)
    tp = int(np.sum((y_true == 1) & (y_pred == 1)))
    fp = int(np.sum((y_true == 0) & (y_pred == 1)))
    fn = int(np.sum((y_true == 1) & (y_pred == 0)))
    tn = int(np.sum((y_true == 0) & (y_pred == 0)))
    return tp, fp, fn, tn

tp, fp, fn, tn = confusion(actual, pred)
print("tp, fp, fn, tn =", (tp, fp, fn, tn))
rows = []
for limit in [150, 200, 220]:
    tp, fp, fn, tn = confusion(actual, log["tool_wear_min"] > limit)
    rows.append({"wear_limit": limit, "precision": tp / max(tp + fp, 1), "recall": tp / max(tp + fn, 1)})
print(pd.DataFrame(rows).round(2))
```

Output

```
tp, fp, fn, tn = (4, 1, 1, 6)
   wear limit  precision  recall
0         150         0.83     1.0
1         200         1.00     0.8
2         220         1.00     0.6
```

The expression `max(tp + fp, 1)` avoids a division by zero when a rule predicts no failure at all.

Check your understanding. A function ends with `return tp, fp, fn, tn`. Which line stores its four results under separate names?

- A. `tp, fp, fn, tn = confusion(y, p)`
- B. `tp = fp = fn = tn = confusion(y, p)`
- C. `[tp] = confusion(y, p)`
- D. `confusion(y, p).unpack()`

Answer: A. The function returns a tuple of four values, and a tuple unpacks into as many names as it has elements.

Discipline challenges

The switch in the notebook loads a classification dataset for several fields. Pick the row of your department and repeat the evaluation, paying attention to class balance and error costs.

Department	Challenge
Food Engineering and Biology	Dry bean varieties from shape features, UCI 602 [6], or rice varieties, UCI 545 [18].
Mechanical Engineering and Industrial	Machine failure in the AI4I 2020 predictive

Department	Challenge
Engineering	maintenance data, UCI 601 [9].
Mechanical Engineering and Textile Engineering	Fault types of steel plates from geometric and luminosity features, UCI 198 [19].
Mining Engineering and Geophysical Engineering	Hazardous seismic bumps in a coal mine, UCI 266 [20, 21].
Physics	Gamma-ray showers versus hadron background in the MAGIC telescope, UCI 159 [22].
Chemistry and Environmental Engineering	Ready biodegradability of chemicals from molecular descriptors, UCI 254 [23].
Electrical and Electronics Engineering	Stability of a simulated decentralised smart grid, UCI 471, without the leaking stability margin [24].
Chemical Engineering and Food Engineering	Good versus ordinary wine from physicochemical tests, UCI 186 [25].
Geological Engineering and Earth Sciences Engineering	Lithofacies from well logs, the SEG facies data of Week 9 [26].
Computer Engineering and Statistics	Compare macro and weighted averages and calibration on any multiclass dataset; discuss when each is appropriate.

Weekly task

Choose a classification dataset from the discipline switch or from DATASETS.md, train at least three classifiers with a stratified split, and report the confusion matrix, per-class precision and recall, the macro F1 and a ROC or precision-recall curve. If the classes are imbalanced, choose a threshold from explicitly stated costs and justify them. Explain the most important features with permutation importance and discuss whether they make physical sense, in about 500 words, citing at least three works from this week's references [3, 4, 8].

Research and report assignment (optional)

Evaluating classifiers for rare engineering events. Review how studies on predictive maintenance, defect detection or hazard prediction evaluate classifiers when events are rare. Compare the metrics, resampling strategies and threshold choices, and assess whether the reported results support the claimed industrial value [4, 5, 8, 20].

Both assignments are optional. During an active semester, the weekly task can be sent together with the learning log exported from the interactive lab, and the research report on its own, to utkukose@sdu.edu.tr or utkukose@gmail.com. They carry no separate weight, but the instructor may take them into account as a discretionary adjustment of the midterm component of the grade. The assessment section of the syllabus gives the details, and research reports follow the report template of the course repository.

References

- [1] Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273-297.
<https://doi.org/10.1007/BF00994018>
- [2] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
<https://doi.org/10.1023/A:1010933404324>
- [3] Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861-874.
<https://doi.org/10.1016/j.patrec.2005.10.010>

- [4] Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
- [5] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321-357. <https://doi.org/10.1613/jair.953>
- [6] Koklu, M., & Ozkan, I. A. (2020). Multiclass classification of dry beans using computer vision and machine learning techniques. *Computers and Electronics in Agriculture*, 174, 105507. <https://doi.org/10.1016/j.compag.2020.105507>
- [7] Koklu, M., & Ozkan, I. A. (2020). *Dry Bean [Dataset]*. UCI Machine Learning Repository, ID 602. <https://doi.org/10.24432/C50S4B>
- [8] Matzka, S. (2020). Explainable artificial intelligence for predictive maintenance applications. In *2020 Third International Conference on Artificial Intelligence for Industries (AI4I)* (pp. 69-74). IEEE. <https://doi.org/10.1109/AI4I49448.2020.00023>
- [9] Matzka, S. (2020). *AI4I 2020 Predictive Maintenance Dataset [Dataset]*. UCI Machine Learning Repository, ID 601. <https://doi.org/10.24432/C5HS5C>
- [10] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56-61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- [11] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>
- [12] Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21-27. <https://doi.org/10.1109/TIT.1967.1053964>
- [13] Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth.
- [14] Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81-106. <https://doi.org/10.1007/BF00116251>
- [15] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794). <https://doi.org/10.1145/2939672.2939785>
- [16] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30* (pp. 4765-4774). <https://arxiv.org/abs/1705.07874>
- [17] Kose, U. (2026). *GEMEX: Geodesic Entropic Manifold Explainability (Version 1.2.2) [Python package]*. Python Package Index. <https://pypi.org/project/gemex/>
- [18] Cinar, I., & Koklu, M. (2019). Classification of rice varieties using artificial intelligence methods. *International Journal of Intelligent Systems and Applications in Engineering*, 7(3), 188-194. <https://doi.org/10.18201/ijisae.2019355381>
- [19] Buscema, M., Terzi, S., & Tastle, W. (2010). *Steel Plates Faults [Dataset]*. UCI Machine Learning Repository, ID 198. <https://doi.org/10.24432/C5J88N>
- [20] Sikora, M., & Wróbel, Ł. (2010). Application of rule induction algorithms for analysis of data collected by seismic hazard monitoring systems in coal mines. *Archives of Mining Sciences*, 55(1), 91-114.
- [21] Sikora, M., & Wróbel, Ł. (2010). *seismic-bumps [Dataset]*. UCI Machine Learning Repository, ID 266. <https://doi.org/10.24432/C5W902>

- [22] Bock, R. (2004). *MAGIC Gamma Telescope [Dataset]*. UCI Machine Learning Repository, ID 159. <https://doi.org/10.24432/C52C8B>
- [23] Mansouri, K., Ringsted, T., Ballabio, D., Todeschini, R., & Consonni, V. (2013). *QSAR biodegradation [Dataset]*. UCI Machine Learning Repository, ID 254. <https://doi.org/10.24432/C5H60M>
- [24] Arzamasov, V. (2018). *Electrical Grid Stability Simulated Data [Dataset]*. UCI Machine Learning Repository, ID 471. <https://doi.org/10.24432/C5PG66>
- [25] Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems*, 47(4), 547-553. <https://doi.org/10.1016/j.dss.2009.05.016>
- [26] Hall, B. (2016). Facies classification using machine learning. *The Leading Edge*, 35(10), 906-909. <https://doi.org/10.1190/tle35100906.1>